

[L] 李硕 · 前端开发工作汇报 [R] 转转 [C]1

前端开发技术报告

校园二手物品交易平台——转转

李硕 · 前端开发

华中农业大学 · 2026 年 7 月

目录

1 概述	3
2 首页是怎么工作的	3
2.1 数据加载流程	3
2.2 商品网格的响应式布局	3
2.3 时间格式化	4
2.4 公告栏滚动	4
3 商品列表页的筛选系统	5
3.1 筛选参数如何在 URL 中保持	5
3.2 前端如何实现分页	5
3.3 分类级联选择器	5
4 商品详情页	6
4.1 HTML 安全过滤	6
4.2 图片画廊	6
5 购物车	6
5.1 全选/取消全选	6
5.2 防抖 (Debounce)	7
5.3 结算流程	7
6 订单列表的双视角切换	7
7 管理后台	8
7.1 整体架构	8
7.2 商品审核页	8
7.3 数据统计页——ECharts 图表	8
8 商品种子数据	9
8.1 数据是怎么生成的	9
8.2 图片是怎么生成的	10
9 数据库：数据是怎么组织的	10
10 前后端如何协作	11
10.1 数据格式约定	11
10.2 接口对接流程	11
10.3 错误处理策略	12

目录	2
11 项目中的具体问题与解决	12
12 总结	13
13 深入：CSS Grid 布局原理	13
13.1 Grid vs Flexbox	13
13.2 auto-fill 和 minmax 的工作原理	13
13.3 绝对单位 vs 相对单位	14
14 深入：MySQL FULLTEXT 全文索引	14
14.1 什么是全文索引	14
14.2 为什么用 FULLTEXT 而不是 LIKE	15
14.3 全文索引的代价	15
15 深入：ECharts 的工作原理	15
15.1 声明式 vs 命令式	15
15.2 ECharts 的渲染管线	15
15.3 为什么折线图用双 Y 轴	16
15.4 Canvas vs SVG	16
15.5 内存泄漏的机制	16
16 深入：MySQL 数据库设计基础	16
16.1 为什么用 InnoDB 引擎	16
16.2 什么是事务	17
16.3 索引的原理	17
16.4 utf8mb4 vs utf8	17
17 深入：前端性能优化	17
17.1 路由懒加载	17
17.2 骨架屏 (Skeleton Screen)	18
17.3 防抖 (Debounce) 和节流 (Throttle)	18
17.4 为什么要把图片下载到本地	18
18 深入：RESTful API 设计	18
18.1 什么是 REST	18
18.2 版本号	19
18.3 统一返回格式	19

1 概述

我在项目中负责前端页面的开发和数据准备工作。具体来说：

- 首页（Hero 横幅、商品网格、统计分析）
- 商品模块（列表搜索筛选、详情展示、发布表单）
- 购物车（选品、改量、结算）
- 订单列表（买家卖家双视角切换）
- 管理后台（5 个管理页面 + 侧边栏布局）
- ECharts 数据可视化图表
- 155 件商品种子数据和图片生成

以下按模块逐一说明做了什么、怎么做的、为什么这么做。

2 前端基础概念速览

在看具体代码之前，先理解几个基础概念。

什么是组件：Vue 中每个 `.vue` 文件就是一个组件。组件就像一个乐高积木——把页面拆分成可复用的小块。比如导航栏是一个组件，商品卡片是另一个组件，它们可以组合在一起构成完整页面。组件化的好处：修改导航栏只需要改一个文件，所有页面自动更新。

什么是 Props 和 Emit：父组件通过 Props（属性）向子组件传递数据，就像父母给孩子东西。子组件通过 Emit（事件）向父组件发送消息，就像孩子告诉父母“我点了一个按钮”。

什么是 v-if 和 v-for：v-if 是条件渲染——“如果条件满足才显示这段 HTML”。v-for 是列表渲染——“对数组中的每个元素，重复这段 HTML”。例如商品网格就是用 `v-for="item in products"` 循环渲染出所有商品卡片。

什么是 CSS：CSS（层叠样式表）控制页面的外观——颜色、大小、位置、动画。CSS Grid 和 Flexbox 是两种现代布局方式，用来决定元素在页面上的排列方式。

3 首页是怎么工作的

3.1 数据加载流程

当你打开首页（/），页面同时发出 4 个请求：

- GET /api/v1/index/statistics → 获取统计数据（用户数、商品数、订单数）
- GET /api/v1/category/list → 获取分类列表（6 个一级 + 20 个二级）
- GET /api/v1/announcement/list → 获取公告列表
- GET /api/v1/product/list?page=1&size=20 → 获取首页商品

这四个请求是并行发出的（不是等一个完成再发下一个），这样可以更快地加载页面。每个请求都有独立的 try-catch，一个失败了不影响其他。

3.2 商品网格的响应式布局

商品卡片排列使用 CSS Grid 布局：

```
.product-grid {  
  display: grid;  
  grid-template-columns: repeat(auto-fill, minmax(240px, 1fr));  
  gap: 20px;  
}
```

通俗解释：让浏览器自动计算一行能放下几个卡片。每个卡片最少 240px 宽，如果屏幕够宽就多放几个，屏幕窄就少放几个。auto-fill 表示自动填充，1fr 表示每个卡片平均分配剩余空间。

3.3 时间格式化

商品列表中的时间是绝对时间（如“2026-07-05 14:30:00”），但展示给用户时应该用相对时间。实现逻辑：

```
function formatTime(t) {  
  const d = new Date(t)           // 解析时间字符串  
  const now = new Date()          // 当前时间  
  const diff = now - d            // 时间差（毫秒）  
  if (diff < 60000) return '刚刚' // < 1分钟  
  if (diff < 3600000) return Math.floor(diff/60000) + '分钟前' //  
    < 1小时  
  if (diff < 86400000) return Math.floor(diff/3600000) + '小时前'  
    // < 1天  
  return d.toLocaleDateString('zh-CN') // 超过1天显示日期  
}
```

3.4 公告栏滚动

公告栏水平滚动用纯 CSS 实现，不需要 JavaScript 定时器。

核心原理：复制一份公告内容放在末尾，然后让整个内容从右到左无限移动。当第一份内容完全滚出屏幕时，它无缝衔接到末尾的副本继续滚动，视觉上形成无限循环。

```
@keyframes noticeScroll {
  0% { transform: translateX(0); }
  100% { transform: translateX(-50%); } /* 移动一半宽度 */
}
.notice-track {
  animation: noticeScroll 40s linear infinite; /* 40秒一圈，匀速，无限 */
}
.notice-board:hover .notice-track {
  animation-play-state: paused; /* 鼠标悬停暂停 */
}
```

4 商品列表页的筛选系统

4.1 筛选参数如何在 URL 中保持

商品列表页有 5 个筛选条件：关键词、分类、价格区间、成色、排序。如果用户筛选后刷新页面，条件不应丢失。

实现方法：把所有筛选参数写入 URL query string。例如：`/product/list?keyword=iPhone&cate`

```
// 当用户改变筛选条件时，更新 URL
function applyFilters() {
  const query = {}
  if (keyword.value) query.keyword = keyword.value
  if (categoryId.value) query.categoryId = categoryId.value
  // ...
  router.push({ query }) // 更新 URL，不刷新页面
}

// 页面初始化时，从 URL 读取筛选条件
onMounted(() => {
  if (route.query.keyword) keyword.value = route.query.keyword
  if (route.query.categoryId) categoryId.value = Number(route.query.categoryId)
})
```

```
// ...  
loadProducts()  
})
```

4.2 前端如何实现分页

分页组件 (Element Plus 的 `el-pagination`) 维护两个关键数据:

- `page`: 当前第几页
- `size`: 每页显示多少条 (默认 20)

每次翻页时, 前端发送 GET `/api/v1/product/list?page=2&size=20`, 后端返回该页的数据和总条数 `total`。前端用 `total` 除以 `size` 算出总页数, 显示在分页组件上。

4.3 分类级联选择器

商品发布时需要选择分类 (如 “电子产品 > 手机”)。使用 Element Plus 的 `Cascader` 组件, 数据来自 `/category/list` 接口, 返回的 JSON 已经是嵌套结构 (父级含 `children` 数组), `Cascader` 可以直接使用。

```
[  
  { "id":1, "name":"电子产品", "children":[  
    { "id":7, "name":"手机"},  
    { "id":8, "name":"电脑"}  
  ]},  
  { "id":2, "name":"书籍教材", "children":[...]}  
]
```

用户选择 “手机” 后, `Cascader` 返回的值是 `[1, 7]` (父级 ID + 子级 ID), 提交表单时取最后一个元素 7 作为 `categoryId`。

5 商品详情页

5.1 HTML 安全过滤

商品描述可能包含 HTML (富文本)。如果不加过滤, 恶意用户可以注入 `<script>alert('攻击')</script>`, 这在其他用户浏览时会被执行 (XSS 攻击)。

过滤实现分三步:

1. `.replace(/<script[>]*>.*?<script>/gi,"")` — `<script>`

1. `.replace(/<style[>]*>.*?<style>/gi,"")` — `<style>`

1. `.replace(/<[>]*>/g,"")` — `HTML`

1. 通过 DOM 的 `textarea.innerHTML = ...; textarea.value` 解码 HTML 实体(如 `&` → `&`)

5.2 图片画廊

商品可以有多张图片 (`product_image` 表)。主图显示一张大图，下方排列缩略图。点击缩略图切换主图，通过改变 `currentImage` 响应式变量的值实现。

如果图片加载失败（网络问题或 URL 失效），`<img>` 标签的 `@error` 事件会触发，显示 Emoji 占位符（如手机分类显示）。

6 购物车

6.1 全选/取消全选

全选状态通过计算属性动态判断：

```
// 当所有项的 selected 都为 true 时，全选 = true
const allSelected = computed(() =>
  list.value.length > 0 && list.value.every(i => i.selected)
)

// 切换全选：把所有项设为相同值
function toggleAll() {
  const newVal = !allSelected.value
  list.value.forEach(i => i.selected = newVal)
}
```

6.2 防抖 (Debounce)

数量调节按钮 (+/-) 如果用户快速连点，会发送大量网络请求。防抖策略：用户每次点击后不立即发送请求，而是等 300ms，如果 300ms 内又点击了，就重新计时。只有最后一次点击后 300ms 内没有新点击，才真正发送请求。

6.3 结算流程

点击”结算”→弹出地址选择对话框→用户选择或添加地址→确认下单→为每个选中商品调用 `POST /api/v1/order` →跳转到支付页面。

地址选择对话框从 `/api/v1/user/address` 获取用户地址列表，默认选中”默认地址”。

7 订单列表的双视角切换

这是订单列表最重要的功能改进。原始代码中 `role` 被写死为 `'buyer'`，卖家永远看不到自己的订单。

修复方案：增加一个 `El-Radio-Group` 切换按钮——”我买的”（`role=buyer`）和”我卖的”（`role=seller`）。切换时重新请求数据：

```
const role = ref('buyer') // 默认显示"我买的"

async function loadOrders() {
  const params = { role: role.value } // 动态切换角色
  if (activeStatus.value) params.status = activeStatus.value
  const res = await orderApi.getList(params)
  list.value = res.data || []
}
```

当切换到”我卖的”并选择”待发货”标签时，页面会显示绿色提示条，引导卖家进入订单详情发货。订单卡片上还有一个突出的”去发货”按钮，点击直接跳转。

8 管理后台

8.1 整体架构

管理后台有独立的布局（`AdminLayout`），使用侧边栏导航。路由配置中设置了 `meta: { requiresAdmin: true }`，路由守卫检查用户角色为非管理员时重定向到首页。

8.2 商品审核页

这是管理后台最复杂的页面。表格展示所有商品，筛选栏支持按状态（待审核/在售/已下架/审核驳回）过滤。操作按钮包括：

- 通过：调用 `PUT /api/v1/admin/product/review/{id}`，状态改为”在售”

- 驳回：同上，状态改为” 审核驳回”
- 下架：调用 `PUT /api/v1/admin/product/{id}/offline`，状态改为” 已下架”
- 编辑：弹出对话框，可修改标题、价格、成色、状态、描述

所有操作都有确认对话框（`ElMessageBox.confirm`），防止误操作。

筛选条件变化时自动重置页码到第一页：用 `watch` 监听筛选参数变化，在回调中设置 `page = 1`。

8.3 数据统计页——ECharts 图表

这是项目中唯一涉及数据可视化的页面，使用 ECharts 5 库。

图表初始化流程：

1. 在模板中放置一个 `<div ref="chartRef">` 作为图表的容器
2. 在 `onMounted` 中获取 DOM 引用，调用 `echarts.init(chartRef.value)` 创建图表实例
3. 调用 `chart.setOption({...})` 设置图表的配置和数据
4. 在 `onUnmounted` 中调用 `chart.dispose()` 销毁实例（释放内存）

饼图配置要点：

```
const option = {
  series: [{
    type: 'pie',
    radius: ['40%', '70%'],    // 环形图（内径40%，外径70%）
    data: [
      { name: '手机', value: 15 },
      { name: '电脑', value: 15 },
      // ...其他分类
    ],
    label: { show: true, formatter: '{b}: {c}件 ({d}%)' }
  ]
}
chart.setOption(option)
```

折线图配置要点：

```
const option = {
  xAxis: { type: 'category', data: ['7/1', '7/2', ...] },    // X轴 日期
  yAxis: [
```

```
{ type: 'value', name: '用户数' },    // 左Y轴
{ type: 'value', name: '订单数' }    // 右Y轴
],
series: [
  { name: '新注册用户', type: 'line', data: [...] },
  { name: '新订单', type: 'line', yAxisIndex: 1, data: [...] }
]
}
```

内存泄漏防护：ECharts实例持有 DOM 引用，组件卸载后如果不手动销毁，实例会一直占用内存。在 SPA 中频繁切换页面会积累大量未释放的图表实例。解决方案是在 onUnmounted 中调用 `chart.dispose()` 和 `removeEventListener`。

9 商品种子数据

9.1 数据是怎么生成的

155 件商品数据通过 SQL INSERT 语句批量写入。每件商品包含：

- 基本信息：标题、描述、价格、原价
- 分类：categoryId (1-25, 对应 6 个一级 + 19 个二级分类)
- 成色：5 个枚举值之一
- 卖家：user_id 随机分配给 17 个用户
- 统计：viewCount、favoriteCount 随机赋值

9.2 图片是怎么生成的

经过 6 轮方案迭代，最终采用 Python Pillow 库自定义生成图片。脚本流程：

```
# 1. 从数据库读取所有商品
for pid, title, cat_id in products:
    # 2. 根据分类选择背景色
    color, label = cat_colors.get(cat_id)

    # 3. 创建 640x480 的图片
    img = Image.new('RGB', (640, 480), rgb)

    # 4. 在图片上绘制文字
    draw = ImageDraw.Draw(img)
```

```
draw.text((320, 240), title, fill="white", anchor="mm")

# 5. 保存为 JPG
img.save(f"{pid}.jpg", quality=90)
```

每张图片都包含了该商品的分类标签和商品名称，确保图文 100% 对应。图片存储在 Docker 容器的 `/data/zhuanzhuan/uploads/products/` 目录中，通过 Nginx 代理由后端提供访问。

为什么不用外部图库：在项目开发过程中，我们尝试了多种外部图片来源（Lorem-Flickr、Picsum、Unsplash、Pexels），但都存在问题——部分图片与商品内容不符、部分 URL 失效、部分服务不稳定。最终的本地生成方案虽然图片是简单的文字 + 色块，但它是 100% 可靠、100% 匹配、0 外部依赖的。

10 数据库：数据是怎么组织的

虽然不是直接写后端，但理解数据库结构对前端开发很重要。本项目的核心数据表：

- **user** 表：存储用户信息。关键字段：username（用户名）、password_hash（BCrypt 加密的密码）、role（user/admin）、status（0 禁用/1 启用）。有软删除（deleted_at）。
- **product** 表：存储商品信息。关键字段：user_id（卖家）、category_id（分类）、title、price、status（待审核/在售/已下架/已售出/审核驳回）。有软删除和 FULLTEXT 全文索引。
- **order** 表：存储订单。关键字段：order_no（唯一订单号）、buyer_id、seller_id、product_id、status（待付款/待发货/待收货/已完成/已取消）。
- **cart** 表：购物车。联合唯一键（user_id, product_id），同一用户同一商品只保留一条。
- **message** 表：站内信。from_user_id（发送方）、to_user_id（接收方）、is_read（0 未读/1 已读）。
- **notification** 表：系统通知。user_id（接收方）、type（order/review/system）、is_read。

其中最重要的设计是 **order** 表的状态机——5 个状态之间有严格的转换规则：待付款 → 待发货 → 待收货 → 已完成，中间任何一步都可以转成“已取消”。每次状态变更都会在 **order_log** 表中记录一条日志。

11 前后端如何协作

11.1 数据格式约定

前后端通过 JSON 格式通信。所有接口返回统一格式：

```
{
  "code": 200,           // 状态码：200 成功，4xx 客户端错误，5xx 服务端
                        错误
  "message": "success",
  "data": {...}         // 实际数据
}
```

前端 Axios 响应拦截器检查 code，如果是 200 就返回 data，否则弹出错误提示。

11.2 接口对接流程

以”发布商品”为例：

1. 前端收集表单数据（标题、分类、价格、描述、图片 ID 列表）
2. 前端通过 Axios 发送 POST /api/v1/product，请求头自动带 Token
3. 后端的 Spring Security 过滤器验证 Token
4. 验证通过后，请求到达 ProductController.publish() 方法
5. Controller 调用 ProductService.publish()，后者开启数据库事务
6. 事务中：插入 product 表 → 更新 product_image 表的 product_id → 设置封面图
7. 返回 JSON 结果给前端
8. 前端收到成功响应，跳转到商品详情页

11.3 错误处理策略

前端对网络错误做了分类处理：

- ECONNABORTED：请求超时（15 秒），提示”检查网络连接”
- 无 response：网络断开，提示”网络连接失败”
- 401：Token 过期，自动刷新 Token

- 403: 权限不足。如果本地有 Token 说明用户可能被删除, 清除登录
- 404: 资源不存在
- 429: 请求太频繁
- 5xx: 服务器错误, 提示” 稍后重试”

12 项目中的具体问题与解决

问题一: ECharts图表在窗口变大变小后不变。原因: ECharts 不会自动响应窗口尺寸变化。解决: 监听 window resize 事件, 回调中调用 chart.resize()。

问题二: 管理后台筛选后跳到空页。原因: 当前在第 5 页, 筛选后数据只有 2 页, 但页码没变。解决: watch 筛选条件变化时重置 page = 1。

问题三: 购物车结算创建多个独立订单。原因: 每个选中商品分别调用 orderApi.create, 如果中间某个失败, 部分订单已创建无法回滚。目前的模拟环境可以接受, 但生产环境应该由后端提供批量创建接口。

问题四: 骨架屏与实际内容高度不一致导致跳动。解决: 骨架屏卡片使用与真实卡片相同的布局结构 (图片区 + 文字区), 确保高度一致。

问题五: 商品图片加载失败破坏布局。解决: 标签的 @error 事件中替换为 Emoji 占位图。

问题六: 首页 4 个请求彼此独立, 其中一个失败不应影响其他。解决: 每个请求都有独立 try-catch, 失败后数据为空但页面不崩溃。

13 总结

这个项目中我写的代码主要解决了以下问题:

- **首页如何展示大量商品:** CSS Grid响应式布局 + 骨架屏加载 + 4 个并行 API 请求
- **筛选条件如何持久化:** URL query string同步, 刷新页面筛选不丢失
- **购物车的状态管理:** 全选/单选用 computed 属性联动, 数量修改用防抖优化
- **卖家如何看到并处理自己的订单:** 订单列表增加” 我买的/我卖的” 切换
- **管理后台的数据可视化:** ECharts饼图 + 折线图, 带 resize 自适应和 dispose 防泄漏
- **商品数据从哪里来:** 155件种子数据 SQL + Python 自定义生成图片, 100% 可靠

14 深入：CSS Grid 布局原理

14.1 Grid vs Flexbox

Flexbox：一维布局。适合排列一行（或一列）中的元素。例如导航栏水平排列、卡片内的文字和按钮垂直排列。

Grid：二维布局。同时控制行和列。适合整个页面的布局、商品网格等。本项目的首页商品网格使用 Grid。

14.2 auto-fill 和 minmax 的工作原理

```
grid-template-columns: repeat(auto-fill, minmax(240px, 1fr));
```

这行 CSS 的背后逻辑：

1. 浏览器先算出容器宽度（比如 1200px）
2. 用 `minmax(240px, 1fr)` 来计算：每个列最少 240px，如果空间够就平均分配
3. `auto-fill` 告诉浏览器：尽可能多地创建列，填不满就留空
4. $1200 / 240 = 5$ 列
5. 如果窗口缩小到 800px： $800 / 240 = 3$ 列（第 4 列不够 240px，换行）

这就是为什么同一段 CSS 在宽屏和窄屏上表现不同——浏览器自动计算列数，实现了真正的响应式布局。

14.3 绝对单位 vs 相对单位

px（像素）：绝对单位，在任何屏幕上都是一样大。**fr**（fraction）：相对单位，表示“剩余空间的 N 分之一”。`1fr 2fr` 表示前者占 1/3，后者占 2/3。

为什么用 fr 而不是 px：如果用固定像素，屏幕变大后卡片不会自动变宽，两侧留白。用 fr，卡片自动填满可用空间。

rem：相对于根元素（<html>）的字体大小。默认 `1rem = 16px`。修改根元素字体大小可以全局缩放所有使用 rem 的元素。

15 深入：MySQL FULLTEXT 全文索引

15.1 什么是全文索引

普通的 `LIKE '%keyword%'` 搜索虽然能用，但是遍历整张表逐行匹配，数据量大时非常慢（ $O(n)$ 复杂度）。全文索引通过建立倒排索引（记录每个词出现在哪些行），

将搜索复杂度降到接近 $O(\log n)$ 。

倒排索引的原理：假设有三件商品：

- 商品 1: "全新 iPhone 手机"
- 商品 2: "二手小米手机"
- 商品 3: "iPhone 手机壳全新"

倒排索引：

- "全新" → 商品 1, 商品 3
- "iPhone" → 商品 1, 商品 3
- "手机" → 商品 1, 商品 2, 商品 3
- "小米" → 商品 2
- "二手" → 商品 2
- "壳" → 商品 3

搜索"iPhone 手机"时，MySQL 查到"iPhone"出现在商品 1 和 3，"手机"出现在商品 1、2、3。取交集得到商品 1 和 3。这比逐行扫描快得多。

15.2 为什么用 FULLTEXT 而不是 LIKE

- LIKE '%iPhone%': 必须扫描每一行的 title 和 description 字段，计算量大
- MATCH(title,description) AGAINST('iPhone')：利用倒排索引直接定位匹配行，速度快几个数量级
- 全文索引还支持自然语言模式、布尔模式（+ 必须包含、-排除、* 通配符）等高级搜索

本项目在 product 表上建立了 FULLTEXT KEY ft_title_desc (title, description)，支持对标题和描述的同时搜索。

15.3 全文索引的代价

索引不是免费的。每次 INSERT 或 UPDATE 都会更新索引，占用额外的磁盘空间。所以只在搜索频繁的列上建全文索引。对于像 view_count 这种只用于排序统计的列，普通 B-Tree 索引就够用了。

16 深入：ECharts 的工作原理

16.1 声明式 vs 命令式

ECharts 是声明式的：你告诉它”我要一个饼图，数据是这些，颜色用这些”，它自动完成绘制。你不需要告诉它”先画一个圆，再画一个扇形，加上标签”。这类似于 Vue 的声明式模板——你说”这个 div 里显示 count”，Vue 自动处理 DOM 更新。

声明式的优势：开发者关注”要什么”而不是”怎么做”；框架负责底层实现和性能优化；代码更简洁、更易维护。**命令式**（如 Canvas API）：灵活但代码量大，每个像素都要手动控制。

16.2 ECharts 的渲染管线

当你调用 `chart.setOption(...)` 后，ECharts 内部执行以下步骤：

1. 解析配置对象，验证必要字段是否存在
2. 数据预处理：对数据进行排序、过滤、聚合
3. 布局计算：根据图表类型和容器大小，计算每个元素的位置和大小
4. 坐标系建立：确定 X 轴和 Y 轴的范围、刻度、标签
5. 图形渲染：使用 Canvas 或 SVG 引擎绘制图形
6. 交互绑定：绑定 hover、click 等事件处理

其中第 3 步（布局计算）是最复杂的。饼图需要计算每个扇区的起始角度和角度范围，折线图需要把所有数据点映射到像素坐标。

16.3 为什么折线图用双 Y 轴

本项目的数据统计页展示两个不同量级的数据：用户注册量（通常几十到几百）和订单成交量（同样几十到几百，但趋势不同）。如果共用一个 Y 轴，两条线的波动幅度可能差异很大，视觉上不友好。

双 Y 轴（左轴和右轴各有自己的刻度）让每条线都在自己的合理范围内展示，更容易看出各自的趋势变化。

16.4 Canvas vs SVG

ECharts 默认使用 Canvas 渲染。Canvas 和 SVG 是两种不同的 Web 图形技术：

Canvas：位图模式。你告诉画笔“在坐标 (100, 200) 画一个圆”，它就真的在那个位置画像素。画完后原来的圆“消失了”（只剩像素）。适合大量图形元素、频繁重绘的场景（如数据可视化）。

SVG：矢量模式。每个图形是一个独立的 DOM 元素（`<circle>`、`<rect>` 等）。你可以用 CSS 修改它的样式、用 JS 绑定事件。适合图形数量不多、需要交互的场景（如图标、按钮）。

ECharts 也支持 SVG 模式（通过 `renderer: 'svg'` 配置），但 Canvas 在大数据量下性能更好。对于本项目的几十个数据点，两种模式几乎没有性能差异。

16.5 内存泄漏的机制

JavaScript 的内存管理依赖垃圾回收（GC）。当一个对象不再被任何变量引用时，GC 会在某个时刻回收它。

ECharts 实例持有对 DOM 元素的引用（通过 `chartRef.value`）、对 Canvas 上下文的引用、对事件监听器的引用。如果不在组件销毁时手动释放这些引用，GC 无法判断这些对象是否还需要。即使页面已经切换走了，ECharts 实例仍然“活”在内存中。

在 SPA 中，用户频繁切换页面会积累大量“僵尸”图表实例，内存占用持续增长。这就是为什么必须在 `onUnmounted` 中调用 `chart.dispose()` 和 `removeEventListener`。

17 深入：MySQL 数据库设计基础

17.1 为什么用 InnoDB 引擎

MySQL 支持多种存储引擎，最常用的是 InnoDB 和 MyISAM。

InnoDB：支持事务（ACID）、行级锁、外键约束。适合需要数据一致性的业务场景。本项目全部使用 InnoDB。

MyISAM：不支持事务、表级锁。读取速度快但不适合频繁写入的场景。全文索引在 MySQL 5.6 之前只有 MyISAM 支持，现在 InnoDB 也支持了。

17.2 什么是事务

事务是一组数据库操作，它们要么全部成功，要么全部失败（原子性）。经典的例子是银行转账：从 A 账户扣款和向 B 账户加款必须同时成功或同时失败，不能出现“扣了 A 的钱但没加到 B”。

本项目在订单创建时使用事务。Spring 的 `@Transactional` 注解让方法内的所有数据库操作在一个事务中执行。任何一步失败，整个事务回滚。

17.3 索引的原理

数据库索引类似于书的目录。没有索引时，查找“数据结构”这本书需要从第一页翻到最后一页（全表扫描）。有了索引，先查目录找到“数据结构”在哪一页，直接翻过去。

B-Tree 索引（MySQL 默认索引类型）：数据按顺序存储，查询复杂度 $O(\log n)$ 。适合范围查询（>、<、BETWEEN）。

Hash 索引： $O(1)$ 常数时间，但只支持等值查询（=）。MySQL 的 Memory 引擎支持 Hash 索引。

为什么不要给所有列都建索引：每个索引都是额外存储空间，写入（INSERT、UPDATE、DELETE）时需要同时更新所有索引。索引太多会拖慢写入速度。只在查询条件经常用到的列上建索引。

17.4 utf8mb4 vs utf8

MySQL 的 utf8 字符集实际只支持最多 3 字节的字符（BMP 平面），不能存储 Emoji（需要 4 字节）。utf8mb4（UTF-8 for Multilingual Plane 4-byte）是真正的 UTF-8，支持所有 Unicode 字符。本项目统一使用 utf8mb4，确保中文、Emoji 都能正确存储和显示。

18 深入：前端性能优化

18.1 路由懒加载

如果不做懒加载，所有页面的 JS 代码在首次访问时全部下载。对于本项目 21 个页面，这会导致首次加载缓慢。路由懒加载将每个页面分离成独立的 JS 文件（chunk），只在真正访问时才下载。Vite 构建时自动处理动态 import 的分割。

18.2 骨架屏（Skeleton Screen）

传统的 loading spinner（转圈圈）让用户感觉“系统在忙”。骨架屏预先画出页面的大致结构（灰色方块代替图片、灰色条代替文字），让用户感觉“内容马上就来”。研究表明骨架屏能显著降低用户的感知等待时间。

实现方式：在数据加载前用 `v-if="!loaded"` 显示骨架屏的灰色占位元素；数据加载完成后 `v-if="loaded"` 切换到真实内容。

18.3 防抖（Debounce）和节流（Throttle）

防抖：在事件触发后延迟执行，如果延迟时间内又触发了，重新计时。适用场景：搜索框输入（用户停止输入后再发请求）、购物车数量修改（停止点击后再同步）。

节流：无论触发多少次，固定时间间隔执行一次。适用场景：滚动事件（每 100ms 检查一次）、窗口 resize。

18.4 为什么要把图片下载到本地

外部 URL 图片有三个问题：加载慢（依赖第三方服务器）、不稳定（URL 可能失效）、不可控（图片内容可能变化）。下载到本地后的优势：加载快（同服务器、无跨域延迟）、稳定（不受外部影响）、可控（内容确定）。CDN 虽然能加速，但对于本项目的 155 张小图片，本地存储完全够用。

19 PPT 演讲文字稿 (约 1 分钟)

以下是我在答辩 PPT 第 8 页（个人分工页）的演讲内容，基于 PPT 上展示的 6 个核心贡献点展开：

各位老师好，我是李硕，在本项目中负责前端页面开发和数据准备工作。

我主要做了五个方面的工作。

第一，首页和商品模块。首页实现了 Hero 横幅、公告栏滚动轮播、分类导航和商品网格。特别说明一下商品网格——用 CSS Grid 的 auto-fill 和 minmax 函数实现了响应式布局，屏幕宽时显示更多列，窄时自动减少。商品列表页实现了关键词搜索、分类筛选、价格区间和排序功能，所有筛选条件通过 URL 参数持久化，刷新页面不会丢失。

第二，购物车和订单列表。购物车实现了全选和单选联动、数量修改防抖同步。订单列表最重要的改进是增加了买家卖家双视角切换——原始代码只显示买到的订单，我增加了切换按钮让卖家也能看到卖出的订单，待发货的订单上还有直接跳转去发货的按钮。

第三，管理后台。我开发了 5 个管理页面——用户管理、商品审核、订单管理、数据统计和公告管理。其中数据统计页使用了 ECharts 库，用饼图展示分类占比，用折线图展示每日趋势。图表做了自适应窗口大小和组件卸载时的内存释放。

第四，数据可视化方面，我特别关注了 ECharts 的内存管理。在 Vue 的单页应用中，页面切换时如果不销毁图表实例，它们会一直占用内存。我在每个图表组件的卸载生命周期中调用了 dispose 方法来释放资源。

第五，商品数据准备。我编写了覆盖全部 20 个分类的 155 件商品种子数据。商品图片经过多次方案迭代，最终采用 Python 脚本自定义生成——每张图片都显示商品名称和分类标签，确保图文 100% 匹配，图片存储在本地服务器，不依赖任何外部服务。

以上就是我的工作内容，谢谢各位老师。

20 深入：RESTful API 设计

20.1 什么是 REST

REST (Representational State Transfer) 是一套 API 设计规范。核心思想：URL 代表资源，HTTP 方法代表操作。

本项目的 API 设计遵循 REST 风格：

- GET /api/v1/product/list —— 获取商品列表
- GET /api/v1/product/123 —— 获取商品 123 的详情
- POST /api/v1/product —— 创建新商品
- PUT /api/v1/product/123 —— 更新商品 123
- DELETE /api/v1/product/123 —— 删除商品 123

这种设计的好处：URL 结构一致、易于理解和文档化、前后端可以独立开发（只要约定好接口格式）。

20.2 版本号

本项目 API 路径中包含 /v1/。版本号的作用：当 API 需要不兼容的变更时，可以发布 /v2/ 版本，旧的 /v1/ 继续保持。这样不会破坏已有的客户端。

20.3 统一返回格式

所有接口返回相同的 JSON 结构：{code, message, data}。前端只需要写一个响应拦截器处理所有接口的返回，不用每个接口单独判断成功/失败。